

SW subsistemas. Inmoov 1.1

Enfoque

El control de los servomotores es un aspecto fundamental en cualquier robot humanoide, especialmente cuando se emplean mecanismos impresos en 3D y servos de alto par para mover piezas grandes y delicadas, como es el caso del InMoov. Un movimiento brusco o demasiado rápido puede provocar sobrecargas, picos de corriente y rotura de elementos mecánicos, por lo que es esencial que los movimientos sean suaves y controlados.

La biblioteca estándar Servo.h de Arduino es la opción más habitual para controlar servomotores mediante señal PWM, ya que permite conectar y manejar múltiples servos en placas como Arduino Uno y Mega. Sin embargo, esta biblioteca no proporciona mecanismos integrados para el control de velocidad o para la generación de rampas de aceleración y deceleración; simplemente posiciona el servo de manera inmediata en el ángulo indicado.

Existen bibliotecas de terceros para Arduino que permiten un control más sofisticado de la velocidad, como

VarSpeedServo de Jeff Korman ([GitHub - netlabtoolkit/VarSpeedServo: Arduino library for servos that extends the standard servo.h library with the ability to set speed, and wait for position to complete](#)), que introduce un parámetro de

velocidad en el movimiento de cada servo. Sin embargo, esta biblioteca tiene un límite de ocho servos gestionados simultáneamente, lo que resulta insuficiente para el caso del InMoov, que requiere controlar hasta nueve servos en el brazo derecho y hasta catorce en la placa Mega. Además, existen otras alternativas más avanzadas, como

VarSpeedServoRA4M1 de Kaled Souky ([VarSpeedServoRA4M1](#)), orientadas a procesadores más potentes que

[Enfoque](#)

[Instalación IDE Arduino en PC](#)

[Desarrollo](#)

[Subsistema 1](#)

[Subsistema 2](#)

Arduino Uno R3 o Mega y, por tanto, no aplicables a este hardware.

Debido a estas limitaciones, se ha optado por un desarrollo a medida para el control de velocidad, totalmente compatible con la arquitectura planteada y escalable al número de servos necesarios.

El software compara la posición actual del servo con la posición objetivo y realiza pequeños incrementos o decrementos a intervalos regulares, interpolando el movimiento en pasos cortos. Este método, conocido como interpolación lineal, permite realizar movimientos suaves a velocidad constante, sin aceleraciones ni frenadas progresivas.

Aunque puede parecer que con este método no existe una rampa de aceleración o deceleración como tal, el propio servo implementa en sus algoritmos internos dichas rampa. En robótica avanzada se emplean perfiles de velocidad trapezoidal o “S-curve”, en los que el movimiento comienza lentamente, acelera progresivamente hasta alcanzar una velocidad de crucero, y finalmente reduce la velocidad de nuevo antes de llegar al destino.

Instalación IDE Arduino en PC

Este bloque instala y lanza Arduino IDE 2.2.1 en Linux desde AppImage, habilita el acceso a puertos serie y resuelve dependencias y permisos necesarios para su correcta ejecución.

```
1 # Crear un directorio para almacenar el Arduino IDE de forma
2 mkdir -p ~/Aplicaciones/arduino-ide
3 cd ~/Aplicaciones/arduino-ide
4
5 # Descargar la versión 2.2.1 del Arduino IDE en formato AppImage
6 wget https://downloads.arduino.cc/arduino-ide/arduino-ide-2.2.1.Linux_64bit.AppImage
7
8 # Dar permisos de ejecución al archivo AppImage
9 chmod +x arduino-ide_2.2.1_Linux_64bit.AppImage
10
11 # Instalar libfuse2, necesario para ejecutar AppImages en Linux
12 sudo apt update
13 sudo apt install libfuse2
```

```
14
15 # Crear un alias temporal para lanzar Arduino desde la te
16 alias arduino='~/Aplicaciones/arduino-ide/arduino-ide_2.2
17
18 # Añadir el alias al archivo ~/.bashrc para que esté disp
19 echo "alias arduino='~/Aplicaciones/arduino-ide/arduino-ide
20
21 # Aplicar los cambios del bashrc actual
22 source ~/.bashrc
23
24 # Añadir el usuario actual al grupo 'dialout' para tener c
25 sudo usermod -aG dialout $USER
26
27 # ⚠ Es necesario cerrar sesión o reiniciar para que el c
28
```

Para llamar a arduino solo hay que hacer desde un terminal:

```
1 arduino
```

Desarrollo

Se utilizan dos subsistemas:

- El **subsistema 1**, basado en una placa Arduino Uno, se encarga del control completo del brazo derecho.
- El **subsistema 2**, construido alrededor de una placa Arduino Mega, tiene asignado el control del brazo izquierdo (aunque en esta fase solo se utiliza su estructura, sin motorización funcional), así como de la cabeza y los movimientos faciales.

El programa de ambos subsistemas implementa el control de los servomotores del robot InMoov a través de sendas placas Arduino, en el contexto de una arquitectura distribuida basada en ROS 2. Los subsistemas utilizan la interfaz XICRO para recibir comandos desde el ordenador central, permitiendo asignar a cada articulación un ángulo objetivo de forma individual. Cada servo está parametrizado con su propio rango de movimiento seguro, una posición de reposo y una velocidad de avance definida por pasos e intervalo de actualización. El movimiento de los servos se realiza de forma interpolada, avanzando o retrocediendo gradualmente hasta alcanzar la posición objetivo, lo que contribuye a la suavidad y protección de la mecánica.

Al recibir un comando con valor cero, el programa interpreta la orden como una petición para mover la articulación a su posición de reposo, sin necesidad de que el sistema central conozca dicho valor. Los ángulos recibidos son siempre limitados al rango de seguridad configurado para cada servo antes de enviarse al motor. El control es robusto frente a valores fuera de rango y puede adaptarse a cualquier servo del conjunto, incluyendo articulaciones grandes o pequeñas. El sistema es flexible y permite ajustar fácilmente tanto la velocidad global de movimiento como los parámetros individuales de cada articulación.

Subsistema 1

En esta versión 1.1 se incorporan sensores propioceptivos al bíceps derecho del robot, concretamente un potenciómetro de realimentación de posición (el propio de servo) y un sensor de corriente, que permiten estimar el estado interno de la articulación y detectar situaciones anómalas de esfuerzo mecánico.

La configuración estática de los servomotores se encuentra centralizada en el archivo `servo_config_inmoov.h` (https://github.com/aalonsopuig/Inmoov_ROS2/blob/main/Arduino/Xicro_subsys1_ID_1/servo_config_inmoov.h), donde se definen los pines PWM, rangos angulares, posiciones de reposo, límites mecánicos, velocidades, parámetros de realimentación y umbrales de protección.

Servo	Topic ROS 2 / XICRO	Pin PWM	Velocidad por defecto	Ángulo de reposo	Ángulo mínimo permitido	Ángulo máximo permitido
THUMB_R	/thumb_fb	D2	50%	180°	106°	180°

	inge r_R					
INDEX _R	/ind ex_f inge r_R	D3	50%	150°	80°	160°
MIDDLE E_R	/mid dle_ fing er_R	D4	50%	150°	65°	166°
RING_ R	/rin g_fi nger _R	D5	50%	160°	70°	162°
PINKY _R	/pin ky_f inge r_R	D6	50%	155°	95°	168°
BICEP _R	/bic ep_R	D8	40%	22°	16°	85°
ROTAT E_R	/rot ate_ R	D9	50%	105°	60°	145°
SHOU LDER_ R	/sho ulde r_R	D10	10%	70°	45°	150°
OMOP LATE_ R	/omo plat e_R	D11	10%	2°	2°	38°

El programa principal se encarga de inicializar de forma segura la alimentación de los grupos de servos, leer la posición inicial del bíceps derecho mediante su sensor de

realimentación, asociar cada servo a su posición de reposo o posición medida, y recibir posteriormente consignas de posición desde ROS 2. Durante la ejecución, el firmware limita las posiciones ordenadas a los rangos seguros definidos, genera movimientos suaves mediante incrementos progresivos y supervisa la corriente del bíceps derecho para detenerlo automáticamente ante una condición de sobreesfuerzo. Además, incorpora una pantalla OLED de diagnóstico básico, actualizada solo bajo demanda o ante fallo, para reducir el consumo de memoria en el Arduino Uno.

Código para el Arduino uno:

(https://github.com/aalonsopuig/Inmoov_ROS2/blob/main/Arduino/Xicro_subsys1_ID_1/Xicro_subsys1_ID_1.ino):

NOTA IMPORTANTE: Debe mencionarse que el subsistema 1 (Arduino uno) está funcionando cerca de sus límites de memoria, por lo que no parece viable añadir nada más a sus funciones. El paso natural siguiente sería migrar a un sistema con mayor disponibilidad de memoria.

Subsistema 2

El subsistema 2 mantiene el código de la versión 1.0 del proyecto, en la que los ángulos se calculan de modo diferente a la v 1.1 del proyecto. Se espera como parte de la versión 1.2 del proyecto, que se actualice el código, pero de momento se mantiene porque al no tener el brazo izquierdo funcional, realmente se mueven solo los componentes de cabeza y cuello.

En esta versión las configuraciones de los servos forman parte del propio fichero del programa y no se han sacado a un .h

Topic	Servo instanc e	Pin Ardui no	Velo c.	Ángul o	Áng ulo	Áng ulo
-------	-----------------------	--------------------	------------	------------	------------	------------

				repos	MIN	MA
thumb_finger_L	s_thumb_finger_L	2	2	144	112	177
index_finger_L	s_index_finger_L	3	2	135	85	150
middle_finger_L	s_middle_finger_L	4	2	135	70	140
ring_finger_L	s_ring_finger_L	5	2	135	80	160
pinky_finger_L	s_pinky_finger_L	6	2	135	90	165
bicep_L	s_bicep_L	8	1	50	50	110
rotate_L	s_rotate_L	9	1	90	80	135
shoulder_L	s_shoulder_L	10	1	90	90	120
omoplate_L	s_omoplate_L	11	1	5	0	20
neck	s_neck	12	2	110	25	170
rothead	s_rothead	13	1	90	50	130
jaw	s_jaw	26	10	9	9	20
eye_x	s_eye_x	22	2	80	70	107
eye_y	s_eye_y	24	2	80	60	100

Nota: eye_y es Pan y eye_x es tilt

Código para el Arduino Mega:

https://github.com/aalonsopuig/Inmoov_ROS2/blob/main/Arduino/Xicro_subsys2_ID_2/Xicro_subsys2_ID_2.ino